



Ahzab al-Shadhili

أحزاب الإمام أبي الحسن الشاذلي

Technical Documentation & Architecture Overview

Flutter / Dart · Android

Package `com.ahzab.ahzab_app` · version 1.0.0+1

github.com/Youssefhassan1717/ahzab-elshazeliya

September 2026

1. What the Application Is

A native Android reading application for the twenty-two **أحزاب** (litanies) of Imam Abu al-Hasan al-Shadhili. It is a *typographic* application first and a data application second: the difficulty is not storing the text but setting classical Arabic devotional prose so that it reads the way it does in a printed **مصحف**-quality edition — justified without ragged gaps, with Qur'anic quotations distinguished from the supplications that surround them.

The corpus is fixed and ships inside the binary. There is no backend, no account, no network dependency at runtime. Everything — search, bookmarks, favourites, theming — runs offline against a single in-memory dataset.

Property	Value
Content	22 ahzab, 131,872 characters of vocalised Arabic
Qur'anic passages identified	131 blocks, typeset in Uthmanic script with ayah numbers
Repeated passages marked	18 (36 markers, nestable)
Release artefact	~21.3 MB APK, <code>android-arm64</code>
Runtime dependencies	None — fully offline

2. Feature Set

2.1 Reading

- **Mushaf typography.** The whole corpus is set in KFGQPC Uthmanic Hafs, the typeface used for printed Qur'ans, at a 0.95× optical size and 2.15 line height.
- **Kashida justification.** Lines are justified by elongating letters (تطويل) rather than stretching word spaces — the correct typographic behaviour for Arabic. See §5.
- **Pinch to zoom.** Continuous font scaling with exponential damping; double-tap restores the default. Justification is suspended mid-gesture and recomputed on release.
- **Qur'anic distinction.** Detected verses are drawn inside ornate brackets ﴿﴾ with per-ayah rosette numerals, and referenced by sura and verse.
- **Repetition marking.** Passages the litany asks the reader to repeat appear in gold brackets, followed by the count — { ثَلَاثًا } — with the numeral itself picked out in gold.
- **Unified separator.** Every du'a boundary uses ﴿﴾ (rub-el-hizb), replacing the asterisks the source text used.

2.2 Navigation and personalisation

- **Diacritic-insensitive search.** Query and corpus are both normalised, so الرحمن matches الرَّحْمَن. Matches are highlighted in place with circular previous/next navigation.
- **Bookmarks.** Anchored by chunk index plus an exact character range, so they survive font-size changes and kashida re-justification.
- **Favourites.** Up to five pinned ahzab, shown in their own section.

- **Light and dark themes** with an animated 250 ms transition.

3. Technology Stack

Layer	Choice	Notes
Language	Dart, SDK <code>^3.11.1</code>	Null-safe, records and pattern matching used throughout
Framework	Flutter, Material 3	<code>useMaterial3: true</code> , colour scheme seeded from the brand green
State management	<code>provider ^6.1.2</code>	Three <code>ChangeNotifier</code> providers; no global singletons for UI state
Persistence	<code>shared_preferences ^2.3.4</code>	Theme, favourites, bookmarks. No database
Monetisation	<code>google_mobile_ads ^5.3.0</code>	Adaptive banner, interstitial, rewarded
Linting	<code>flutter_lints ^6.0.0</code>	
Testing	<code>flutter_test</code>	Layout-invariant tests over the real corpus
Build toolchain	Gradle KTS, JVM 17	<code>com.ahzab.ahzab_app</code>

3.1 Typefaces

Family	Role
KFGQPC Uthmanic Hafs	Body text of every hizb and all Qur'anic passages
Amiri (Regular, Bold)	Headings, ornaments, bracket glyphs
Scheherazade New (4 weights)	Interface body text
Aref Ruqaa, El Messiri, Reem Kufi, Cairo, Rakkas	Display and decorative use

4. Architecture

The project follows a flat, feature-oriented layout. There is no service locator and no repository abstraction — the dataset is a compile-time constant, so indirection would buy nothing.

```
lib/
  main.dart          // binding init, runApp
  app.dart           // MultiProvider, themes, RTL Directionality
  core/
    kashida.dart      // Arabic justification engine (265 lines)
    arabic_normalizer.dart // diacritic folding + match mapping (171)
    smooth_scroll_physics.dart
    theme/ app_colors.dart, app_theme.dart
  data/
    ahzab_data.dart    // the corpus, with inline markup (254)
  models/ hizb_part.dart
  providers/ theme_ / favorites_ / bookmarks_provider.dart
  screens/
    intro/ intro_screen.dart // (720)
    home/ home_screen.dart + 3 widgets
    detail/ detail_screen.dart // (1415)
            widgets/content_body.dart // the renderer (1182)
    about/, support/
  services/ ads_bootstrap.dart, interstitial_, rewarded_ad_service.dart
  widgets/ ad_banner.dart, animated_list_item.dart, islamic_icon.dart

tool/ 17 offline Dart scripts – corpus analysis and correction
test/ 3 test files
```

4.1 State

Provider	Holds	Preference key
ThemeProvider	Light/dark mode	theme_mode
FavoritesProvider	Set of hizb ids, max 5	favorites_v3
BookmarksProvider	Bookmarks per hizb, lazily loaded and cached in memory	bookmarks_v1_<hizbId>

4.2 The data model

```
class HizbPart {
  final String id; // 'bahr', 'barr', 'lutf', ...
  final String title; // حزب البحر
  final String? subtitle;
  final String content; // vocalised Arabic + inline markup
}
```

5. The Kashida Justifier

Arabic is not justified by widening spaces. It is justified by *elongating the connecting strokes between letters* — inserting `U+0640` (tatweel). Flutter has no such facility, so the app implements it.

5.1 Algorithm

1. Lay the paragraph out once with a `TextPainter` and read back the line boxes.
2. For each line that is neither the last of its paragraph nor under the fill threshold, measure the shortfall against the column width.
3. Enumerate legal elongation spots — positions between a forward-joining letter and a letter that accepts a join. Elongation is only valid where the script actually connects.
4. Distribute tatweels across those spots by iterative measurement, respecting per-spot and per-word caps so no single word absorbs a whole line's slack.
5. Emit an offset table alongside the new string so any index in the source can be mapped into the justified text.

5.2 Why the offset table matters

Bookmarks, search highlights and the Qur'an/repeat styling are all **character ranges**. Inserting tatweel shifts every index after the insertion point — and the number inserted changes with font size and screen width. Without `KashidaText.mapIndex`, a bookmark would drift every time the reader zoomed. This was the root cause of an early bug where brackets appeared to detach from the passages they enclosed.

5.3 Tuning

Constant	Value	Purpose
<code>_minFillRatio</code>	0.55	Below this, a line is treated as end-of-paragraph and left ragged
<code>_maxPerSpot</code>	2	Ceiling on elongation at any single join
<code>_maxPerWord</code>	4	Stops one word swallowing the line's slack
<code>_cacheLimit</code>	32	LRU keyed on text, size, family, letter spacing, width

6. The Markup Language

The corpus carries three inline markers. They are stripped before display; each one contributes a styled range instead. Using a marker rather than a parallel index table keeps the annotation attached to the text through every edit.

Marker	Meaning	Rendered as	Count
<code>§Q§ref text§Q§</code>	Qur'anic passage	Ornate brackets, mushaf text, ayah rosettes	131 blocks
<code>§B§ ... §b§</code>	Repeated passage (nestable)	Gold round brackets	18 pairs
<code>§SECTION§name§SECTION§</code>	Sub-hizb heading	Ornamental band with a centred label	5

6.1 Rendering pipeline

```
raw content
  | collapse blank lines / runs of spaces
  ▼
_parseMarkup → plain text + four range lists
               (verses, frames, inner frames, ornaments)
  ▼
KashidaJustifier.resolve(text, style, width, skipRanges:)
  | — returns justified text + offset table
  ▼
remap every range through KashidaText.mapIndex
  |
  ▼
flatten overlapping layers into disjoint spans
  | outer → inner, later layers merged over earlier
  ▼
TextSpan tree → RichText
```

Layer flattening. A Qur'anic verse can sit inside a repeated passage, which can sit inside another repeat, and a repeat count may overlap both. Naively applying the ranges in order loses the inner style. The renderer therefore collects every range boundary, walks the resulting elementary intervals, and merges the styles that cover each one — so a verse keeps its mushaf face even when nested two levels deep.

7. Qur'an Detection

The Qur'anic quotations in the source were unmarked, unvocalised inconsistently, and spelled in modern orthography rather than the mushaf's رسم. An offline tool (`tool/mark_quran.dart`) locates them against the full text of the Qur'an and rewrites each one from the mushaf.

7.1 Method

1. **Corpus.** All 6,236 ayat are fetched once and cached (`tool/quran.json`). Basmala prefixes on the first ayah of each sura are stripped, as are standalone pause marks.

2. **Seed index.** Every 4-gram of the Qur'an is indexed by its *consonantal skeleton* — long vowels removed for words above three letters. This is what lets السماوات in the litany find السموات in the mushaf.
3. **Expansion.** A seed grows outward while each pair of words is “the same word” under a bounded Levenshtein distance (1 for short words, 2 otherwise).
4. **Snap to ayah boundaries.** The span is trimmed to the largest range of *complete* ayat it contains. Half-quoted verses are deliberately left as plain text.
5. **Merge and bridge.** Adjacent runs of the same sura are merged; a short ayah lying between two matches is bridged if the intervening words check out.
6. **Rewrite.** The matched region is replaced with the mushaf text plus per-ayah numerals, wrapped in a §Q§ marker.

7.2 Failure modes that were found and fixed

Defect	Effect
Standalone waqf marks ۞ treated as words	Every passage crossing one stopped dead — the single largest cause of missed verses
Hamza-above (U+0654) not folded	Words such as يَايُنَيَا never matched
Exact-match seeding	Rasm/orthography differences broke matching before expansion could start
Backward-only expansion	Cursor failed to advance — infinite loop
Spans crossing an existing marker	Rewriting would have deleted the marker; such spans now shrink or are skipped

Result. Coverage rose from 32 marked passages to **131**. Roughly 108 quotations remain unmarked by design: they are partial verses, or verses quoted without their narrative opening (قال), and the project's rule is that only complete ayat are presented as Qur'an.

8. Screens

Screen	Behaviour
Intro 720 lines	Staggered entrance animation — basmala, ornamental rules, title, du'a — over roughly 3.7 s, then hands off to Home. Four controllers drive fade, stagger, a slow decorative loop and the exit.
Home 338 lines	Favourites section above the full list. Search debounced at 250 ms and matched against normalised titles and full text, with a per-card match count. Ad initialisation is deferred to the first post-frame callback.
Detail 1,415 lines	The reader. Content is split on section markers into keyed chunks. Base font size is derived from device width (24 pt at 411 dp, clamped 22–34) and pinch scaling is clamped 10–44 pt. Search results scroll to 35 % of the viewport; the app bar hides on scroll and snaps at the edges.
About	Biography of the Imam and a description of the collection.
Support	Optional rewarded-ad flow for readers who wish to support the app.

8.1 Performance work

- **Ad SDK initialisation moved off startup.** `MobileAds.initialize()` spins up a WebView on the Android UI thread; called from `main()` it stalled the intro animation. It now runs from Home's first post-frame callback.
- **Full-screen compositing layers removed.** Three stacked `Opacity` layers and an animated full-screen radial gradient were deleted from the intro.
- **Justification suspended during gestures.** The kashida pass is far too heavy to repeat per frame, so an identity transform is used while a pinch is in flight.

9. Offline Tooling

Seventeen Dart scripts under `tool/` operate on the corpus. They are deliberately Dart rather than shell: PowerShell silently corrupted Unicode escapes in replacement strings during early work, writing literal `\u0060C` sequences into the text.

Script	Purpose
<code>fetch_quran.dart</code>	Downloads and caches all 6,236 ayat
<code>mark_quran.dart</code> 397 lines	The detector described in §7
<code>quran_coverage.dart</code>	Reports near-complete quotations that remain unmarked
<code>why.dart</code>	Diagnostic — replays seeding and expansion for one passage
<code>audit.dart</code>	Finds stray brackets, blank lines, split words, repeat labels
<code>tidy.dart</code>	Applies corpus-wide corrections: word joining, separator unification
<code>corrections.dart</code>	Targeted edits, including quoting named passages in full
<code>check_markers.dart</code>	Marker integrity — balance and absence of illegal nesting
<code>frame_repeats.dart</code> , <code>reframe.dart</code> , <code>nest_barr.dart</code>	Repeat-marker construction
<code>dump_hizb.dart</code> , <code>slice.dart</code> , <code>find.dart</code>	Inspection helpers for a file whose lines run to tens of thousands of characters

10. Testing

Test	Assertion
<code>kashida_test.dart</code>	Tatweel insertion preserves the source; short strings are left untouched; mapped ranges still bracket the same word
<code>kashida_width_test.dart</code>	Across all 22 ahzab , no justified token exceeds a 360 dp column. This test caught a real defect — a 432 px token produced by unbounded per-word elongation
<code>widget_test.dart</code>	Smoke test on app start-up

Static analysis runs clean of errors across `lib/`.

11. Build and Release

```
flutter build apk --release --target-platform android-arm64
└─ build/app/outputs/flutter-apk/app-release.apk  ~21.3 MB
└─ copied to releases/ and committed
└─ git push → distributed as a raw GitHub URL
```

Setting	Value
Application id	<code>com.ahzab.ahzab_app</code>
Version	1.0.0+1
Java / JVM target	17
SDK levels	Flutter defaults (<code>flutter.minSdkVersion</code> etc.)
Icon tree-shaking	Material icons reduced 1,645 KB → 4.5 KB (99.7 %)

Outstanding before store submission. The release build is still signed with the debug keystore — `android/app/build.gradle.kts` carries a `TODO` to that effect. A release signing config must be added before publishing to Google Play.

12. The Corpus

#	Id	Title	#	Id	Title
1	<code>bahr</code>	حزب البحر	12	<code>hijb</code>	حزب الحجب
2	<code>barr</code>	حزب البر	13	<code>hifz</code>	حزب الحفظ
3	<code>lutf</code>	حزب اللطف	14	<code>hamd</code>	حزب الحمد
4	<code>wadhifa</code>	الوظيفة الشاذلية	15	<code>khalwa</code>	حزب الخلوة
5	<code>ikhfa</code>	حزب الإخفاء	16	<code>sawn</code>	حزب الصون
6	<code>shakwa</code>	حزب الشكوى	17	<code>fath</code>	حزب الفتح
7	<code>nasr</code>	حزب النصر	18	<code>falah</code>	حزب الفلاح
8	<code>ad3eya</code>	حزب الأدعية	19	<code>kifaya</code>	حزب الكفاية
9	<code>ishraq</code>	حزب الإشراق	20	<code>najat</code>	حزب النجاة
10	<code>ayat</code>	حزب الآيات	21	<code>nour</code>	حزب النور
11	<code>tawassul</code>	حزب التوسل	22	<code>rizq_hirasah_afw</code>	الرزق والحراسة والعفو

13. Design Language

Token	Light	Dark
Accent	#0F6B3E emerald	#C8A96E gold
Background	#F8FBF8	#0A1A0F
Surface	#FFFFFF	#152A1B
Primary text	#143A26	#E8F4EA
Secondary text	#6B6B6B	#B0CCBA

The accent is theme-dependent by design: gold reads as illumination against the dark ground, while emerald carries the same emphasis on paper-white without glare. Every accented element — Qur'anic brackets, repeat brackets, repeat counts — follows this single token.